Flowchart

Binary Tree Node Counting Using C++ : flowcharts, symbols, decision logic and walkthrough

1. Introduction to the Flowchart

A flowchart is a diagram that shows the logic of a program as a sequence of standard symbols joined by arrows. It helps to understand the program before writing code, to explain it to others, and to find mistakes in the logic. This document gives three flowcharts for the Binary Tree Node Counting program: the **main program**, the recursive **countNodes** function, and an **iterative (queue)** version. Each one is followed by a block-by-block explanation.

2. Flowchart Symbols Used

Symbol	Name	Meaning	Used for
Rounded oval	Terminal	Start or end of the program or function	Start, End, Stop, Return
Rectangle	Process	An operation or calculation	Initialize counters, total++, leaf++, internal++, oneChild++
Diamond	Decision	A Yes / No question with two exits	Node is NULL?, Leaf node?, One child only?, Queue empty?
Arrow	Flow line	Direction in which control moves	Connects all blocks
Purple rectangle	Recursive / call block	A call to another function (or to itself)	count(left); count(right), countNodes(root)

In the diagrams below, **green** rectangles are processes, **purple** shapes are decisions or function calls, and **grey** rounded boxes are start / end points.

3. Rules Followed While Drawing

- Every flowchart has exactly one Start and at least one End.
- Flow goes from top to bottom (and left to right for branches).
- Every decision diamond has one entry and exactly two labelled exits (Yes / No).
- Arrows never cross through other blocks; loops return with a bent arrow.
- Each block contains a short statement, not a full sentence.
- Branches that do the same next work are merged before continuing.

4. Flowchart A : Main Program

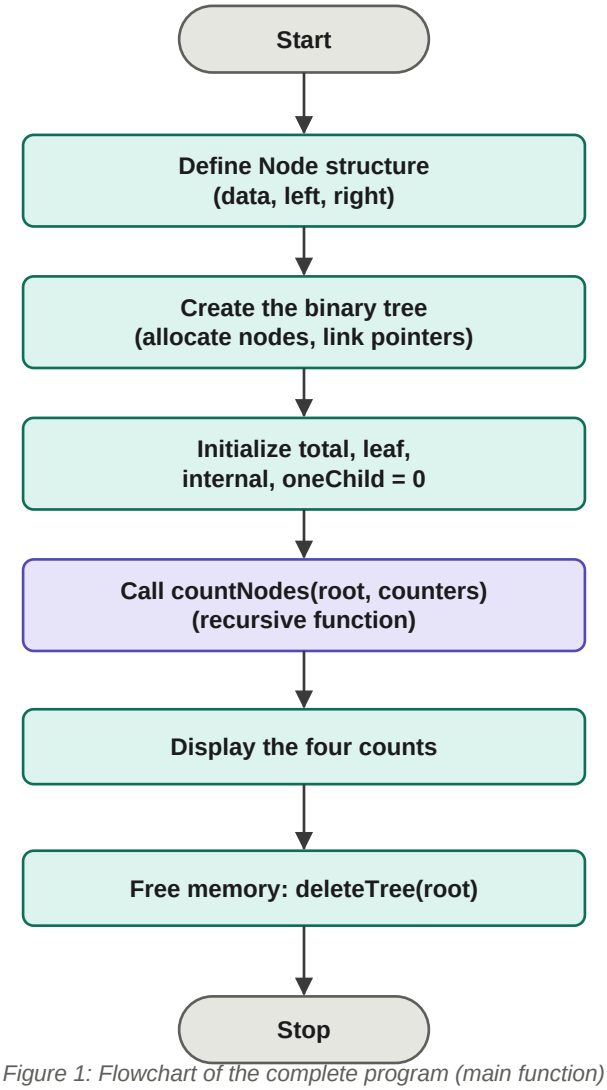

Figure 1: Flowchart of the complete program (main function)

Block	What it does
Start	Program execution begins at main().
Define Node structure	Declares struct Node with data, left and right so that nodes can be created.
Create the binary tree	Allocates each node with new, then links nodes through left and right pointers to form the required tree.
Initialize counters	Sets total, leaf, internal and oneChild to 0 before counting begins.
Call countNodes	Passes the root and the four counters (by reference) to the recursive function. Flowchart B explains this block in detail.
Display the four counts	Prints the final values using cout.
Free memory	deleteTree(root) releases every node using a postorder traversal.
Stop	Program ends.

5. Flowchart B : Recursive countNodes Function

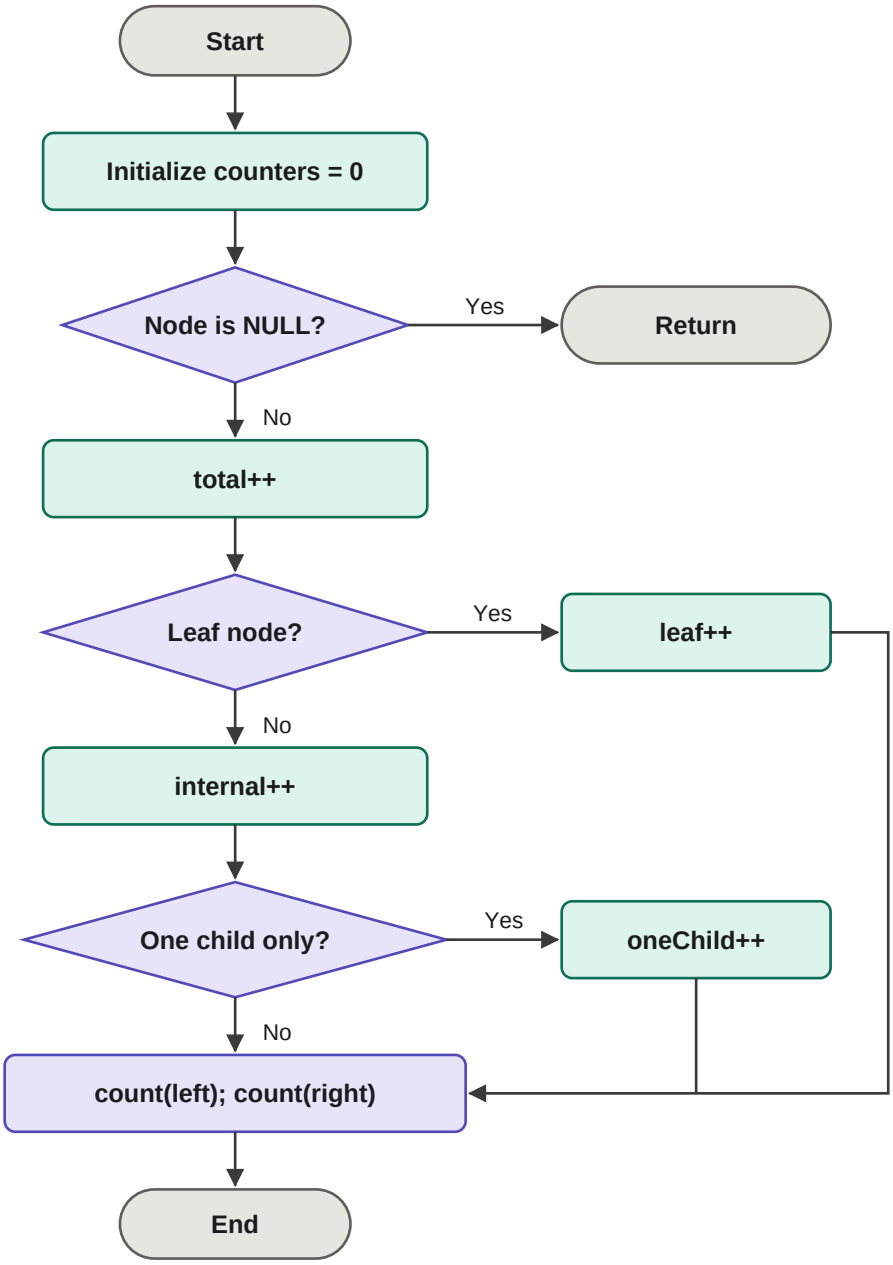

Figure 2: Flowchart of the recursive function countNodes(node)

The flowchart is read from the top. Each call of the function starts at **Start**; when the function reaches **End** or **Return**, control goes back to the place where it was called.

6. Block-by-Block Explanation of Flowchart B

Block	Type	Explanation
Start	Terminal	A call of countNodes(node) begins.
Initialize counters = 0	Process	Done once, in main, before the first call. The counters are shared by all calls (passed by reference).
Node is NULL?	Decision	Base case test. Yes: this subtree is empty, so return. No: a real node exists, continue.
Return	Terminal	Ends this call and goes back to the caller without counting anything.
total++	Process	Counts the current node. Runs for every non-NULL node.
Leaf node?	Decision	Tests whether left == NULL and right == NULL. Yes: the node has no children.
leaf++	Process	Increases the leaf counter. The flow then joins the recursive call block.
internal++	Process	Reached on the No path of the leaf test, so the node has at least one child.
One child only?	Decision	Tests whether exactly one of left and right is NULL. Yes: one child. No: two children.
oneChild++	Process	Increases the one-child counter, then joins the recursive call block.
count(left); count(right)	Call	The function calls itself for the left subtree and then for the right subtree. Each call follows this same flowchart.
End	Terminal	The call finishes and control returns to the caller.

7. Decision Logic Table

The two decisions 'Leaf node?' and 'One child only?' together separate all three kinds of nodes.

left == NULL	right == NULL	Path taken in flowchart	Counters changed
Yes	Yes	Not NULL -> total++ -> Leaf? Yes -> leaf++ -> recurse	total, leaf
Yes	No	Not NULL -> total++ -> Leaf? No -> internal++ -> One child? Yes -> oneChild++ -> recurse	total, internal, oneChild
No	Yes	Not NULL -> total++ -> Leaf? No -> internal++ -> One child? Yes -> oneChild++ -> recurse	total, internal, oneChild
No	No	Not NULL -> total++ -> Leaf? No -> internal++ -> One child? No -> recurse	total, internal

8. Walkthrough of the Flowchart on an Example Tree

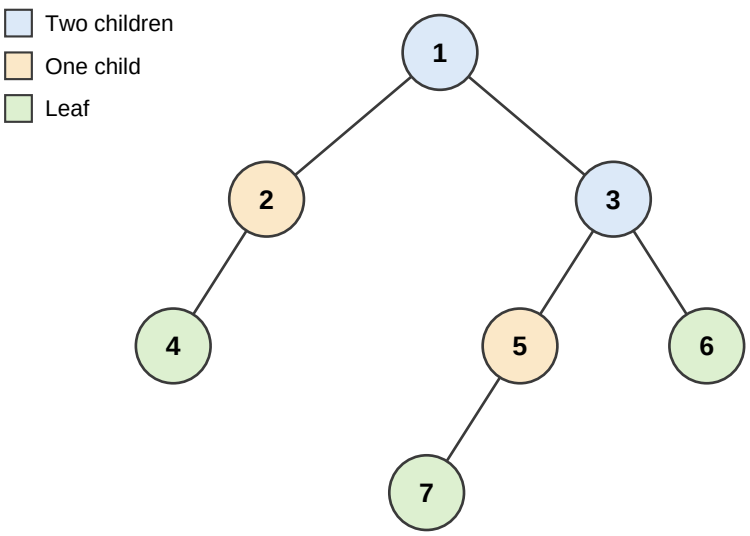

Figure 3: Example binary tree

Example tree used in all documents: node 1 is the root with children 2 and 3; node 2 has only a left child (4); node 3 has children 5 and 6; node 5 has only a left child (7).

Node	Path through Flowchart B	Result
1	NULL? No -> total++ -> Leaf? No -> internal++ -> One child? No -> count(2), count(3)	Two children
2	NULL? No -> total++ -> Leaf? No -> internal++ -> One child? Yes -> oneChild++ -> count(4), count(NULL)	One child
4	NULL? No -> total++ -> Leaf? Yes -> leaf++ -> count(NULL), count(NULL)	Leaf
3	NULL? No -> total++ -> Leaf? No -> internal++ -> One child? No -> count(5), count(6)	Two children
5	NULL? No -> total++ -> Leaf? No -> internal++ -> One child? Yes -> oneChild++ -> count(7), count(NULL)	One child
7	NULL? No -> total++ -> Leaf? Yes -> leaf++ -> two NULL calls	Leaf
6	NULL? No -> total++ -> Leaf? Yes -> leaf++ -> two NULL calls	Leaf
Any NULL pointer	NULL? Yes -> Return	Not counted

Final counters: total = 7, leaf = 3, internal = 4, oneChild = 2.

9. How Recursion Appears in a Flowchart

A flowchart does not draw a separate copy of the diagram for every call. Instead, the block *count(left); count(right)* means 'run this same flowchart again for the left child, wait until it finishes, then run it again for the right child'. The computer remembers where to continue by storing each unfinished call on the **call stack**. The Yes exit of 'Node is NULL?' is what finally stops the repeated calls.

10. Flowchart C : Iterative Version (Queue)

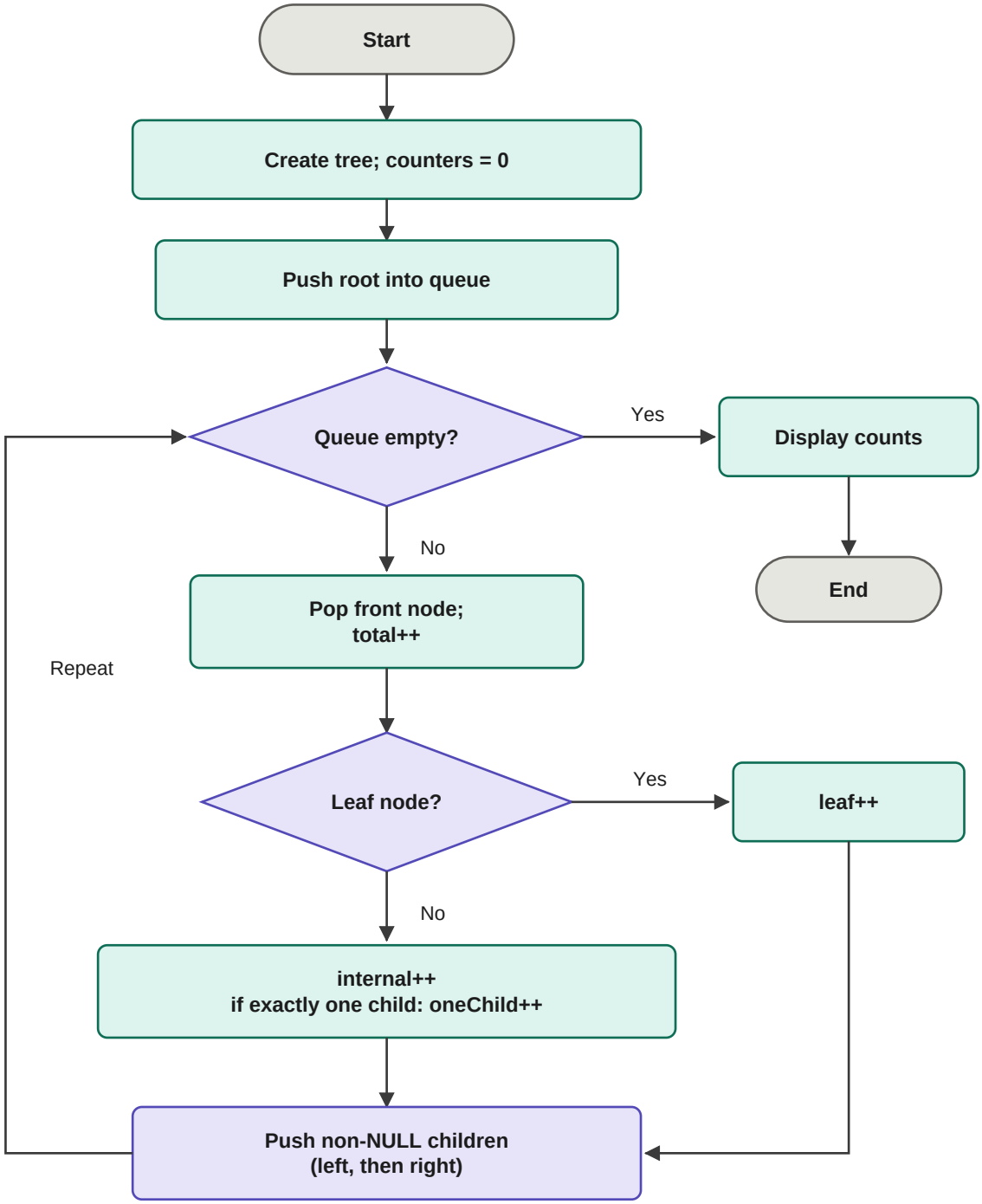

Figure 4: Flowchart of the non-recursive node counting using a queue

Block	Explanation
Create tree; counters = 0	Builds the tree and sets all four counters to zero.
Push root into queue	The queue starts with the root node; the nodes waiting to be processed are kept in it.
Queue empty?	Loop condition. Yes: all nodes are processed, so display the counts. No: process the next node.

Block	Explanation
Pop front node; total++	Takes the oldest waiting node out of the queue and counts it.
Leaf node?	Yes: increase leaf. No: the node is internal.
internal++ / oneChild++	Internal counter increases; if exactly one child exists, oneChild also increases.
Push non-NULL children	Adds the existing left and right children to the queue so they are processed later. The arrow then returns to 'Queue empty?'.

11. Comparison of Flowchart B and Flowchart C

Point	Flowchart B (recursive)	Flowchart C (iterative)
Repetition done by	Function calling itself	A loop that runs until the queue is empty
Stop condition	Node is NULL	Queue is empty
Memory used	Call stack, O(h)	Queue, O(w)
Diagram shape	Straight flow with a call block	Contains a loop-back arrow

12. Common Flowchart Mistakes

- Missing the NULL check, so the recursion never stops.
- Drawing a decision diamond with only one exit or with unlabelled exits.
- Forgetting to merge the leaf++ and oneChild++ branches before the recursive call.
- Using a process rectangle for a question (it must be a diamond).
- Leaving the loop-back arrow out of the iterative flowchart.

13. Summary

The flowchart makes the algorithm easy to follow: first check for NULL, then count the node, then use two decisions to classify it, and finally repeat the same steps for the left and right subtrees. Three diamonds and a recursive call block describe the whole function.